# Shipwreck Management System

## 1.0Introduction

## 1.0Functional Requirements

Eco Credible is a well-established leading shipwreck Store and has many branches in the state of New York. The shipwreck store has the following business activities related to the ship that must be automated.

1. Add Ship Details
2. Modify Ship Details
3. List All Ship Details

You need to implement the shipwreck module of the Shipwreck Management application. The client wishes to have restful web services for adding ship details, modifying the details of ship information like ship name, description, depth, Latitude, Longitude and yearDiscovered. Display the ship information based on the requirement. Help them to automate the above process by developing Rest Service using Maven.

## 2.0 Technical Specifications

The provided ShipController which is a RestController, should be created with the following services:

**Services:**

| Request Method | Request Url | Description |
|---|---|---|
| Post | /addShip | This service should add the ship details by using the addShip method of the ShipServiceImpl |
| Put | /updateDepth/{shipID}/{depth} | This service should update the depth of the existing ship by using the updateDepth of the ShipServiceImpl |
| Get | /getShipWithGoodCondition | This service should retrieve the list of existing Ship details which have Good condition by using getShipWithGoodCondition of the ShipServiceImpl |
| Get | /getShipWithFairCondition | This method should return the list of Ship details that have a Fair condition by using getShipWithFairCondition of the ShipServiceImpl. |

IShipService service;

It should be injected inside the Controller via annotations

## Microservice Specification:

- Spring boot application should contain all the REST services implementation as per the case studyspecification.
- The Spring rest service application should have the application name as "`shipwreckstore`" in the application.properties.
- Register the spring rest service application with the Eureka server which should run in port 8761.

## API Gateway Server:

- Implement API Gateway using Cloud API Gateway
- Must contain a routing information for the below URI mapping to the actual serviceimplementation that is running in the port 9091
    - /addShip URI mapping,
    - /updateDepth/{shipID}/{depth} URI mapping
    - /getShipWithGoodCondition URI mapping and
    - /getShipWithFairCondition URI.

- The API gate way server port (8777) must be specified in the "application.properties" file.
- The service must be accessible through API gateway using small-case url.

***Note:***

**Do not specify absolute url of the rest service in the Gateway properties file. Rather use service-id to specify the name of the application(Rest service) .That is in Gateway, to specify the application nameuse the below property in application.properties.**

**spring.application.name = shipwreckstore**

### Service Layer

Refer to the IShipService interface provided as part of the code skeleton. The ShipServiceImpl which is provided as part of the code skeleton has to realize all the methods in the IShipService interface

**ShipServiceImpl** class should be configured via annotation as Service

| ShipServiceImpl | | | |
|---|---|---|---|
| **Attribute Name** | **Attribute Type** | **Access Specifier** | **Constraints** |
| shipList | List<Ship> | private | |

| Method in Service | Description | Exception |
|---|---|---|
| addShip | This method should add the ship details | If a Ship already exists with the |

| | | |
|---|---|---|
| | to the shipList and return the Ship object.<br><br>Use SLF4J and log the success message as "**Ship with shipID<<shipID>> added successfully**" | same shipID, then throw a user defined exception<br><br>InvalidShipException with the message "**Ship with <<shipID>>already exists!**"<br><br>Use SLF4J and log this as an error message |
| updateDepth | This method accepts shipID and depth as arguments. It should iterate the shipList and replace the existing depth of the ship with the depth parameter based on the shipID passed as the parameter.<br><br>Use SLF4J and log the success message as "**Ship <<shipID>> depth updated successfully to <<new depth>>**" | If no ship exists in that shipID, throw ShipNotAvailableException with the message<br><br>"**Ship with <<shipID>> not available to update the depth**"<br><br>Use SLF4J and log this as an error message |
| getShipWithGoodCondition | This method should iterate the shipList and retrieve the list of ships whose condition is good among the ships in the list.<br><br>Use SLF4J and log the success message as "**Get Ships with good condition is successfully done**" | |
| getShipWithFairCondition | This method should iterate the shipList and retrieve the list of ships whose condition is fair among the ships in the list.<br><br>Use SLF4J and log the success message as "**Get Ships with fair condition is successfully done**" | |

To perform **logging**, you are provided with logger.xml file. In **ShipServiceImpl** when the success / error messages are logged using SLF4J, they get entered into logs/mylogs.log file in the project directory.

## Exception Class

Refer to the **InvalidShipException** and **ShipNotAvailableException** class provided as part of the code skeleton. This class should extend the Exception class. Provide the needed annotation so that it handles HttpStatus as NOT_FOUND

## Model Class

Refer to the Ship class provided as part of the class diagram/code skeleton, use appropriate annotation to generate getter/setter and a no argument constructor.

| Attribute | Data type |
|---|---|
| shipName | String |
| shipID | String |
| description | String |
| condition | String |
| depth | Integer |
| latitude | Double |
| longitude | Double |
| yearDiscovered | LocalDate |

yearDiscovered will be in "yyyy-MM-dd" format. Set this format for Json

**Note:**

- **Use Lombok to generate getters and setters for the attributes**
- **Also generate all-argument constructor and no argument constructor using lombok**
- **Do not change the datatype or the attribute name provided as part of the code skeleton**

## BusinessValidation

### Inbuilt Validation

| Rule | Message when validation fails |
|---|---|
| Ship name should not be empty | Required Ship Name |
| Description should not be empty | Required Description |
| Condition should not be empty | Required Condition |
| Depth should not be NULL | Depth should not be negative |
| Latitude should not be NULL | Latitude should not be null |
| Longitude should not be NULL | Longitude should not be null |
| Year Discovered should not be NULL | YearDiscovered should not be null |

Perform the above Validations using annotations that are available in jakarta.validation.constraints package

### Incorporate Exception and validations as ResponseEntity

To incorporate the messages related to Exception and validations (both custom and inbuilt validations)as ResponseEntity, you are provided with the following classes

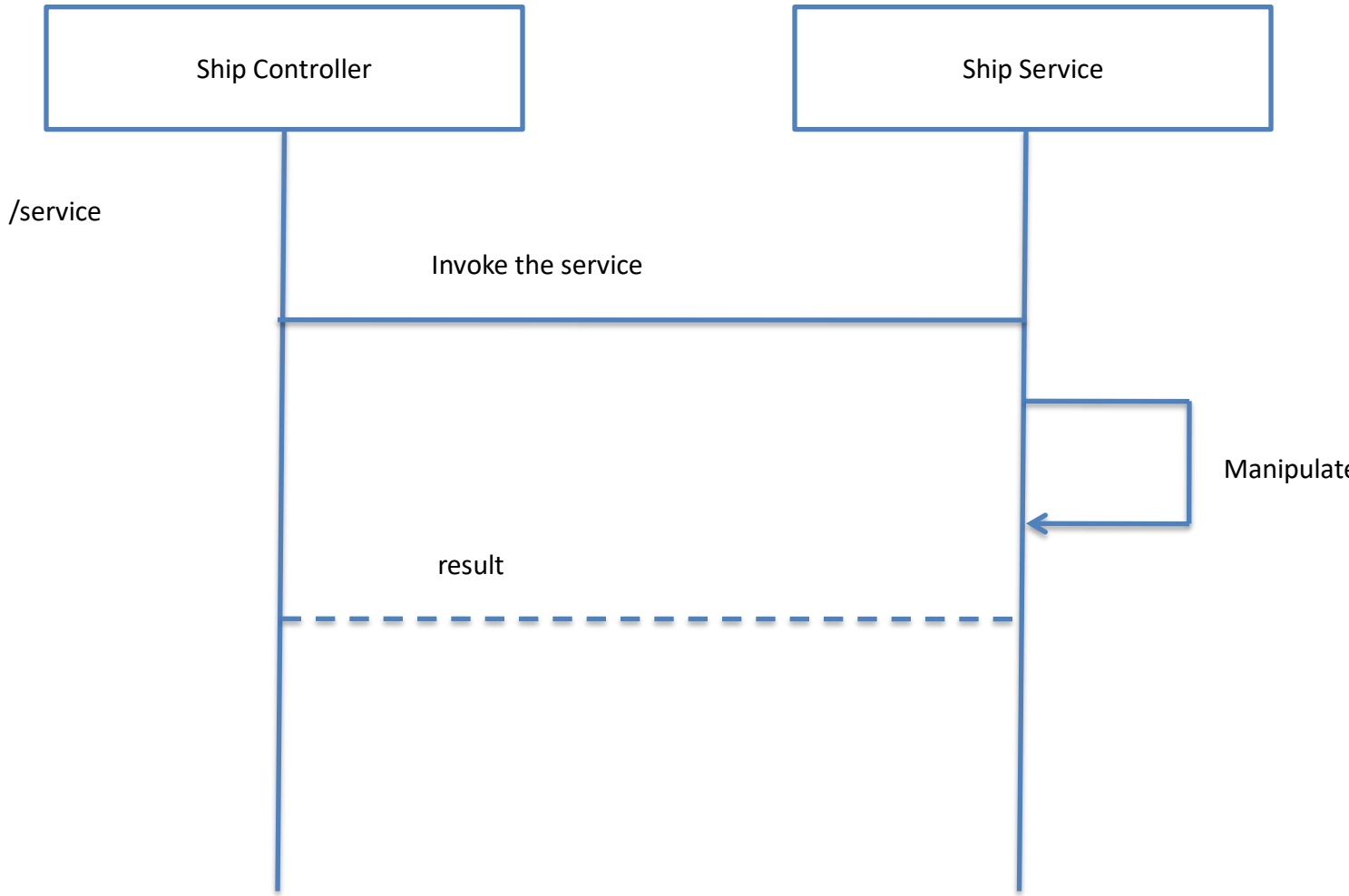
- ExceptionResponse class that represents what attributes needs to be there in the customized response error message. Make use of this class to send the customized response error message.
- CustomizedResponseEntityExceptionHandler class to handle all the exceptionsthat has occurred in the application(both user defined and pre defined).

| **Method Name** | **Explanation** |
|---|---|
| handleAllExceptions | This method should generally handle all the exceptions.<br>In ResponseEntity set the HTTPStatus as ***INTERNAL_SERVER_ERROR.*** |
| handleNotFoundException | This method should handle and provide customized error message using Exception Response class for user defined exception ShipNotAvailableException.<br>In ResponseEntity set the HTTPStatus as ***NOT_FOUND.*** |
| handleInvalidException | This method should handle and provide customized error message using Exception Response class for user defined exception InvalidShipException. In ResponseEntity set the HTTPStatus as ***NOT_FOUND.*** |
| handleMethodArgumentNotValid | This method should handle and provide customized error message using Exception Response class for both pre-defined and user defined validations.<br>In ResponseEntity set the HTTPStatus as ***BAD_REQUEST.*** |

**Note :**

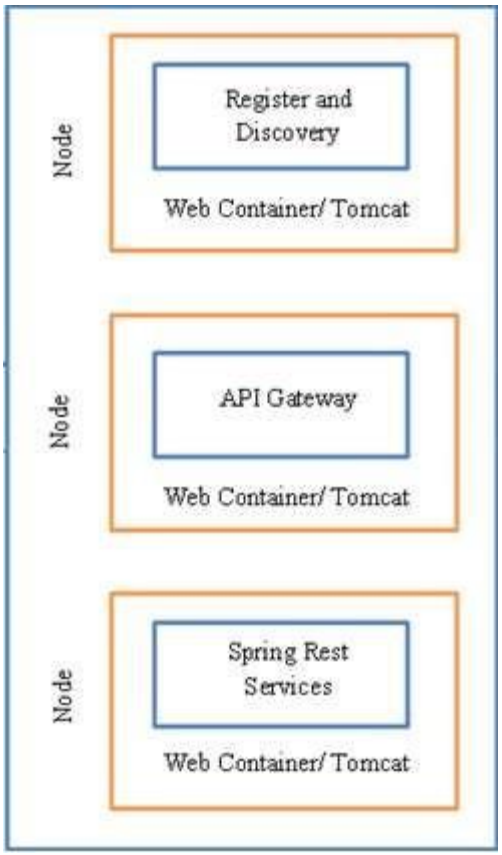
- **In CustomizedResponseEntityExceptionHandler class, log all the error messages using SLF4J.Use Lombok annotations for logging.**

## 3.0 Process Flow




- The Client invokes the required service.
- Controller invokes the method of the ShipServiceImpl.
- ShipServiceImpl performs the service and returns the data back.
- IShipService has to be injected into the ShipController.
- Use appropriate annotations for performing the validations.
- To perform **logging**, you are provided with the log4j.xml file.
- Use **Lombok** to create getters and setters in Ship, ShipServiceImpl and ExceptionResponse class.
- Configure logging using Slf4j annotation with Lombok in ShipServiceImpl and CustomizedResponseEntityExceptionHandlerclass.
- When multiple validations fail, concatenate all messages as a single string and store it in the details attribute of the ExceptionResponse object.

## Architecture diagram:




## 4.0 Microservices implementation Guidelines

For Microservices implementation, we will be implementing 3 projects.

- Spring Cloud (Eureka-Server) for service registry and discovery
- Spring Boot for REST service creation
- Spring Cloud gateway for API Gateway

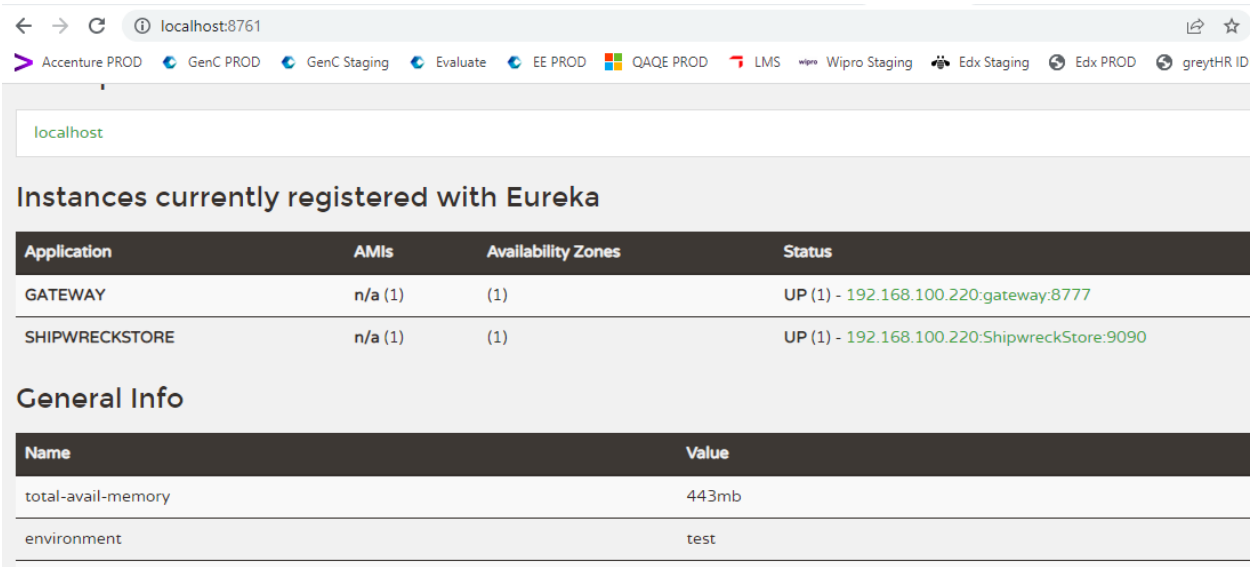
**Project 1:**
This project should be the Eureka-Server acting as the registry. Run this server in port 8761

**Project 2:**

This project should be a spring boot application containing all the REST services implemented as per therequirements stated in the case study.

These services must be registered with Eureka-Server.

Run the services in port 9091.Services should get automatically registered with the Eureka RegistryAfter the service is registered,

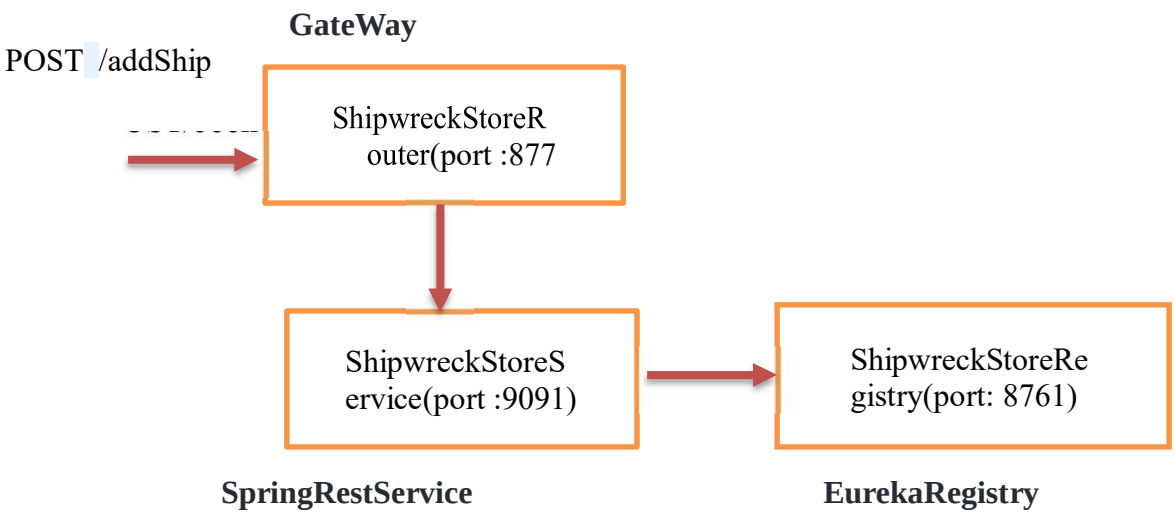



**Note:**

- **Applicationname should be given as shipwreckstore as specified in the case study.**
- **Eureka Registry should run only in port 8761**
- **You can change application port number ie 9091 if required.**

**Project 3:**

This must be a Spring cloud gateway project that contains the routing implementation to the actual servicesrunning in port 9091. Run this in port 8777

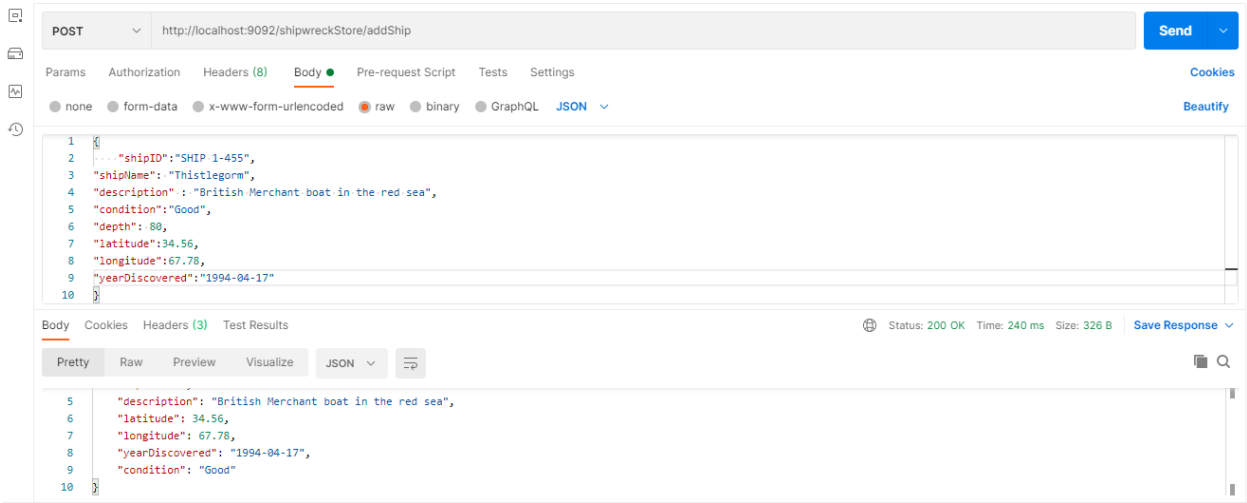



## 4.0 Overall Design Constraints

1) **Do not change the property name given in the application.properties files, you can change the value and you can include additional property if needed.**
2) **In the pom.xml you are provided with all the dependencies needed for developing the application. Do not change or add new dependencies**
3) **Use the service type and the service names as expected in the specification**
4) Adhere to the design specifications mentioned in the case study.
5) Do not change or delete the class/method names or return types that are provided to you as part of the base code skeleton.
6) Services can be tested using external tools like postman by the associate.
7) **Please make sure that your code does not have any compilation errors while submitting your case study solution.**
8) **Your code will not get evaluated if the code skeleton is altered.**

# 5.0 Output –Sample Screen shots in Postman

1. **Rest service for /shipwreckStore/addShip with valid Ship details**



1. **Rest service for /shipwreckStore/addShip with empty shipName and depth**

POST http://localhost:9092/shipwreckStore/addShip

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
"shipID":"SHIP 1-7",

"description" : "British Merchant boat in the red sea",
"condition":"Good",
"depth": -80,
"latitude":34.56,
"longitude":67.78,
"yearDiscovered":"1994-04-17"
}
```

Body Cookies Headers (4) Test Results Status: 400 Bad Request Time: 8 ms Size: 354 B

Pretty Raw Preview Visualize JSON

```
{
    "timestamp": "2022-01-21",
    "message": "validation fails Required Ship Name, Depth should not be negative, Required valid ship ID, ",
    "details": "uri=/shipwreckStore/addShip",
    "errorCode": "Bad Request"
}
```

2. **Rest service for /shipwreckStore/updateDepth/{shipID}/{depth} with valid shipID and depth**

PUT http://localhost:9092/shipwreckStore/updateDepth/SHIP 1-455/100

Params Authorization Headers (7) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Body Cookies Headers (3) Test Results Status: 200 OK Time: 6 ms Size: 327 B Sa

Pretty Raw Preview Visualize JSON

```
{
    "shipID": "SHIP 1-455",
    "shipName": "Thistlegorm",
    "depth": 100,
    "description": "British Merchant boat in the red sea",
    "latitude": 34.56,
    "longitude": 67.78,
    "yearDiscovered": "1994-04-17",
    "condition": "Good"
}
```

3. **Rest service for /shipwreckStore/getShipWithGoodCondition**

GET http://localhost:9092/shipwreckStore/getShipWithGoodCondition

Params Authorization Headers (6) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Body Cookies Headers (3) Test Results Status: 200 OK Time: 13 ms Size: 329 B

Pretty Raw Preview Visualize JSON


```
[
    {
        "shipID": "SHIP 1-455",
        "shipName": "Thistlegorm",
        "depth": 100,
        "description": "British Merchant boat in the red sea",
        "latitude": 34.56,
        "longitude": 67.78,
        "yearDiscovered": "1994-04-17",
        "condition": "Good"
    }
]
```


## 4. Rest service for /shipwreckStore/getShipWithFairCondition

GET http://localhost:9092/shipwreckStore/getShipWithFairCondition

Params Authorization Headers (6) Body Pre-request Script Tests Settings

Body Cookies Headers (3) Test Results Status: 200 OK Time: 7 ms Size: 555 B

Pretty Raw Preview Visualize JSON


```
        "shipName": "SS Yongala",
        "depth": 50,
        "description": "A luxury passenger ship wrecked on the great barrier reef",
        "latitude": 84.56,
        "longitude": 107.78,
        "yearDiscovered": "1994-08-24",
        "condition": "Fair"
    },
    {
        "shipID": "SHIP 7-555",
        "shipName": "Javaship",
        "depth": 60,
        "description": "Programming ship to develop secure applications",
        "latitude": 84.56,
        "longitude": 107.78,
        "yearDiscovered": "1991-05-15",
        "condition": "Fair"
    }
]
```


## 5. Rest service for /shipwreckStore/updateDepth/{shipID}/{depth} with Invalid shipID and depth

PUT http://localhost:9092/shipwreckStore/updateDepth/SHI789/-100

Params Authorization Headers (7) Body Pre-request Script Tests Settings

Body Cookies Headers (3) Test Results Status: 404 Not Found Time: 10 ms Size: 306 B

Pretty Raw Preview Visualize JSON


```
{
    "timestamp": "2022-01-21",
    "message": "Ship with SHI789 not available to update the depth",
    "details": "uri=/shipwreckStore/updateDepth/SHI789/-100",
    "errorCode": "Not Found"
}
```

## 6. Rest service for /shipwreckStore/addShip with duplicate shipID

POST http://localhost:9092/shipwreckStore/addShip

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
{
    "shipID":"SHIP 1-455",
"shipName": "Thistlegorm",
"description" : "British Merchant boat in the red sea",
"condition":"Good",
"depth": 80,
"latitude":34.56,
"longitude":67.78,
"yearDiscovered":"1994-04-17"
}
```

Body Cookies Headers (3) Test Results

Status: 404 Not Found Time: 8 ms Size: 276 B

Pretty Raw Preview Visualize JSON

```
{
    "timestamp": "2022-01-21",
    "message": "Ship with SHIP 1-455 already exists!",
    "details": "uri=/shipwreckStore/addShip",
    "errorCode": "Not Found"
}
```

# PART -II

## INTER MICROSERVICE CONFIGURATION

The information about the agency is provided by the ShipwreckStoreGreeting microservice. The ShipwreckStoreGreeting microservice has the below service method. This service method is present in theGreetingController class

| Request Method | Request Url | Method in Controller | Description |
|---|---|---|---|
| Get | /welcome | greeting | This service will display the information about the agency. |

The solution for the ShipwreckStoreGreeting is provided as part of the code skeleton, you just need to usethis microservice in ShipwreckStoreApplication microservice.

The ShipwreckStoreApplication microservice is responsible for using the welcome service present in the ShipwreckStoreGreeting microservice. The ShipwreckStoreApplication has the below service. This service ispresent in the ApplicationController which is a RestController.

| Request Method | Request Url | Method in Controller | Description |
|---|---|---|---|
| Get | /greet | retreiveInfo | This service will display the information about the ship by invoking the welcome service of the ShipwreckStoreGreeting Microservice.<br><br>Note: Use inter microservice communication – Feign Client |

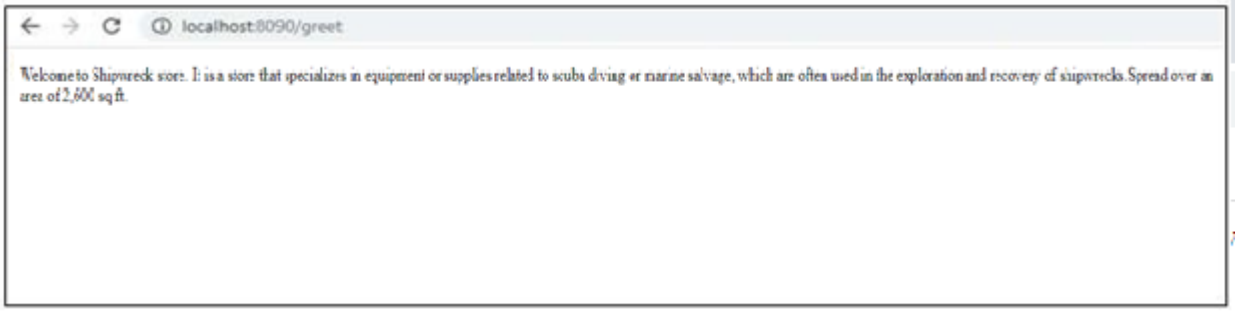
The below proxy interface named ServiceProxy is provided in the ShipwreckStoreApplication Microservice. Use appropriate annotation in this proxy for performing inter microservice communication.

The ServiceProxy should be injected in the ApplicationController. Multiple clients will be accessing the greet service present in the ShipwreckStoreApplication Microservice, so ensure the load balancing is doneaccordingly using client side load balancer.

When the client invokes the greet service, there can be possibilities where the remote service(welcome service of ShipwreckStoreGreeting microservice) might not be available. Handle this failure by using circuit breaker. Also use fallback method in the circuit breaker to display the appropriate message.

# Output –Sample Screen shots in postman

## 1. Rest service for /greet




2. Rest service for /greet when the /welcome service is unavailable in ShipwreckStoreGreeting Microservice